

LBM-PINN: High-Performance Thermal
Simulation
Technical Roadmap

William Lien Chin Wei

February 2026

Contents

1	Phase 1: The Principal Foundation	1
1.1	Week 1: Infrastructure & Scaffolding	1
1.1.1	Modern CMake Architecture	1
1.1.2	RAII and Memory Management	1
1.2	Week 2: The Physics Core (D2Q9)	1
1.2.1	Lattice Equilibrium	1
1.2.2	Collision and Boundary Conditions	2
2	Phase 2: Performance Engineering	3
2.1	Week 3: CPU High-Performance Computing	3
2.2	Week 4: NVIDIA GPU Acceleration (CUDA)	3
2.2.1	Kernel Optimization	3
2.2.2	Profiling & Roofline Analysis	3
3	Phase 3: The L6 Strategic Edge	4
3.1	Week 5: Automation & Analytics	4
3.2	Week 6: Physics-Informed AI (PINN)	4
4	Thermal Diffusion: Numerical Implementation	5
4.1	3D Heat Diffusion Equation	5
4.2	Simplification: 1D Steady-State Conduction	6
4.3	Extension to 2D: Point-Iterative vs. Line-Implicit Methods . .	6
4.3.1	The Point-Iterative Baseline (Jacobi)	7
4.3.2	The Line-Implicit Architecture (LBL TDMA)	7
4.4	Algorithmic Verification and Profiling	8

Chapter 1

Phase 1: The Principal Foundation

1.1 Week 1: Infrastructure & Scaffolding

1.1.1 Modern CMake Architecture

Implementation of `FetchContent` for `GTest/Benchmark` and definition of targets: `core_lib`, `cpu_backend`, and `gpu_backend`.

1.1.2 RAII and Memory Management

Development of the `DeviceBuffer<T>` class for zero-leak `cudaMalloc` management. Implementation of 64-byte aligned allocators for AVX-512 compatibility.

1.2 Week 2: The Physics Core (D2Q9)

1.2.1 Lattice Equilibrium

The equilibrium distribution is implemented as:

$$f_i^{eq}(\rho, \mathbf{u}) = \omega_i \rho \left(1 + \frac{3(\mathbf{e}_i \cdot \mathbf{u})}{c^2} + \frac{9(\mathbf{e}_i \cdot \mathbf{u})^2}{2c^4} - \frac{3\mathbf{u}^2}{2c^2} \right) \quad (1.1)$$

1.2.2 Collision and Boundary Conditions

Implementation of the BGK Single Relaxation Time (SRT) model and interchangeable "Bounce-Back" vs "Zhu-He" objects via Dependency Injection.

Chapter 2

Phase 2: Performance Engineering

2.1 Week 3: CPU High-Performance Computing

Reorganization of memory from AoS (Array of Structures) to SoA (Structure of Arrays). Optimization of thread affinity using `OMP_PLACES` and `OMP_PROC_BIND`.

2.2 Week 4: NVIDIA GPU Acceleration (CUDA)

2.2.1 Kernel Optimization

Moving the D2Q9 kernel to the GTX 1650. Implementation of tiled streaming via Shared Memory to reduce Global Memory traffic.

2.2.2 Profiling & Roofline Analysis

Using Nsight Compute to perform Roofline Analysis, determining if the kernel is Memory Bound or Compute Bound.

Chapter 3

Phase 3: The L6 Strategic Edge

3.1 Week 5: Automation & Analytics

Integration of Pybind11 to bind the C++ solver to Python. Automated grid convergence studies to prove 2nd-order accuracy.

3.2 Week 6: Physics-Informed AI (PINN)

Integration of the physics loss function into the neural network:

$$\mathcal{L}_{total} = \mathcal{L}_{data} + \lambda \mathcal{L}_{physics} \quad (3.1)$$

Training surrogate models to predict thermal fields based on Reynolds and Rayleigh numbers.

Chapter 4

Thermal Diffusion: Numerical Implementation

4.1 3D Heat Diffusion Equation

The differential form of the heat diffusion equation for an isotropic medium is:

$$\rho c_p \frac{\partial T}{\partial t} = \frac{\partial}{\partial x} \left(k \frac{\partial T}{\partial x} \right) + \frac{\partial}{\partial y} \left(k \frac{\partial T}{\partial y} \right) + \frac{\partial}{\partial z} \left(k \frac{\partial T}{\partial z} \right) + \dot{q} \quad (4.1)$$

We integrate the heat diffusion equation over the control volume $\Delta V = \Delta x \Delta y \Delta z$ centered at node P (see Figure ??):

$$\int_{\Delta t} \iiint_V \rho c_p \frac{\partial T}{\partial t} dV dt = \int_{\Delta t} \oint_A (k \nabla T) \cdot d\mathbf{A} dt + \int_{\Delta t} \iiint_V \dot{q} dV dt \quad (4.2)$$

Applying a fully implicit time-stepping scheme and evaluating face fluxes via central differencing, we arrive at the general discrete algebraic form:

$$a_P T_P = a_W T_W + a_E T_E + a_S T_S + a_N T_N + a_B T_B + a_T T_T + b \quad (4.3)$$

Where the coefficients are defined to maintain physical consistency across 3D space:

- $a_E = \frac{k_e \Delta y \Delta z}{\delta x_{PE}}, \quad a_W = \frac{k_w \Delta y \Delta z}{\delta x_{WP}}$
- $a_N = \frac{k_n \Delta x \Delta z}{\delta y_{PN}}, \quad a_S = \frac{k_s \Delta x \Delta z}{\delta y_{SP}}$

- $a_T = \frac{k_t \Delta x \Delta y}{\delta z_{PT}}, \quad a_B = \frac{k_b \Delta x \Delta y}{\delta z_{BP}}$
- $a_P^0 = \frac{\rho c_p \Delta V}{\Delta t}$ (Temporal inertia)[cite: 1]
- $a_P = \sum a_{nb} + a_P^0 - S_P \Delta V$ [cite: 1]
- $b = S_u \Delta V + a_P^0 T_P^0$ [cite: 1]

4.2 Simplification: 1D Steady-State Conduction

To verify the TDMA kernel today, we collapse the 3D general form under two assumptions:

1. **Steady-State:** $\frac{\partial T}{\partial t} \rightarrow 0$, therefore $a_P^0 = 0$ [cite: 1].
2. **1D Heat Flow:** $\Delta y, \Delta z \rightarrow 1$ and $a_N, a_S, a_T, a_B = 0$ [cite: 1].

The system collapses to a tridiagonal form suitable for the TDMA solver:

$$a_P T_P = a_W T_W + a_E T_E + b \quad (4.4)$$

In our code implementation (`solver_1d.f90`), this maps to the matrix indices i :

$$\underbrace{-a_W}_{a_i} T_{i-1} + \underbrace{a_P}_{b_i} T_i + \underbrace{-a_E}_{c_i} T_{i+1} = \underbrace{b}_{d_i} \quad (4.5)$$

4.3 Extension to 2D: Point-Iterative vs. Line-Implicit Methods

Transitioning from 1D to 2D steady-state conduction introduces significant computational bottlenecks if treated purely explicitly. For a 2D uniform grid ($\Delta x = \Delta y$), the continuous Laplace equation discretizes to a 5-point stencil. We evaluate two distinct numerical architectures for solving this system, prioritizing cache coherency and convergence acceleration.

4.3.1 The Point-Iterative Baseline (Jacobi)

The explicit Jacobi method updates each node independently using the localized average of its orthogonal neighbors:

$$T_{i,j}^{new} = \frac{a_E T_{i+1,j} + a_W T_{i-1,j} + a_N T_{i,j+1} + a_S T_{i,j-1} + b}{a_P} \quad (4.6)$$

For a uniform, homogeneous grid (where $\Delta x = \Delta y$ and conductivity k is constant), the coefficients collapse to $a_{nb} = 1$ and $a_P = 4$, reducing the system to the localized average of its orthogonal neighbors:

$$T_{i,j}^{new} = \frac{1}{4} (T_{i+1,j} + T_{i-1,j} + T_{i,j+1} + T_{i,j-1}) \quad (4.7)$$

While mathematically simple and "embarrassingly parallel" (ideal for naive GPU porting), this Point-Iterative approach is highly inefficient. It possesses a spectral radius near 1.0, resulting in painfully slow propagation of boundary information. Furthermore, it requires an out-of-place memory update (a "Ping-Pong" secondary buffer) to avoid data race conditions, effectively doubling the memory footprint and saturating DRAM bandwidth.

4.3.2 The Line-Implicit Architecture (LBL TDMA)

To drastically reduce the iteration count and memory overhead, we pivot to a Line-By-Line (LBL) implicit sweep. By treating the horizontal (x) direction implicitly and the vertical (y) direction explicitly, we collapse the 2D field into a series of coupled 1D systems:

$$-a_W T_{i-1,j} + a_P T_{i,j} - a_E T_{i+1,j} = a_N T_{i,j+1} + a_S T_{i,j-1} + b \quad (4.8)$$

For our verified Laplace case on a uniform grid, substituting the collapsed coefficients yields:

$$-T_{i-1,j} + 4T_{i,j} - T_{i+1,j} = T_{i,j-1} + T_{i,j+1} \quad (4.9)$$

This maps perfectly to our highly optimized core TDMA solver (Equation 4.5). The coefficients become $a_W = 1$, $a_E = 1$, $a_P = 4$, with the source term $d_i = T_{i,j-1} + T_{i,j+1}$.

HPC Ramifications: The Line-Implicit approach propagates boundary conditions across an entire row instantaneously. Crucially, it allows for *in-place* memory updates (effectively becoming a Line-Gauss-Seidel method).

By eliminating the $O(N \times M)$ "Ping-Pong" vector swap, we shift the algorithm from being memory-bandwidth bound to compute-bound, a vital prerequisite for scaling on modern architectures.

4.4 Algorithmic Verification and Profiling

Rigorous Test-Driven Development (TDD) via Google Test was utilized to profile the numerical efficiency of both solvers on a 50×50 Cartesian grid, subject to a stringent L_∞ norm tolerance of 10^{-7} .

Solver Architecture	Max Iterations	Residual at Iter 1000	Memory Pattern
Jacobi (Point-Iterative)	$> 5,400$	1.40×10^{-3}	Out-of-place (Ping-Pong)
LBL TDMA (Line-Implicit)	1,560	9.86×10^{-6}	In-place (No allocation)

Table 4.1: Convergence profiling of 2D thermal solvers targeting 10^{-7} tolerance.

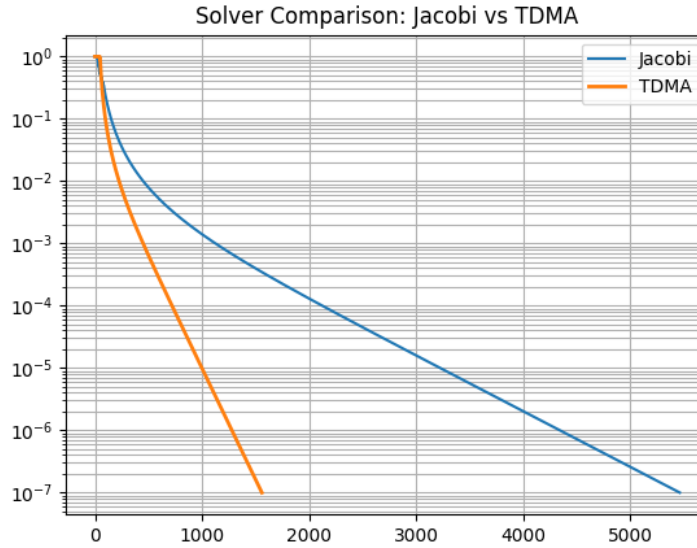


Figure 4.1: Solver convergence comparison: L_∞ residual vs. iteration count for the Jacobi baseline and Line-Implicit TDMA architecture.

The empirical data demonstrates a **3.4x reduction in total iterations** when utilizing the Line-Implicit TDMA sweep. More significantly, at the 1000-iteration mark, the TDMA architecture achieves a residual two orders of magnitude lower than the Jacobi baseline. This validates the mathematical superiority of the implicit formulation and establishes a highly robust numerical core for future predictive health and advanced thermal simulation platforms.

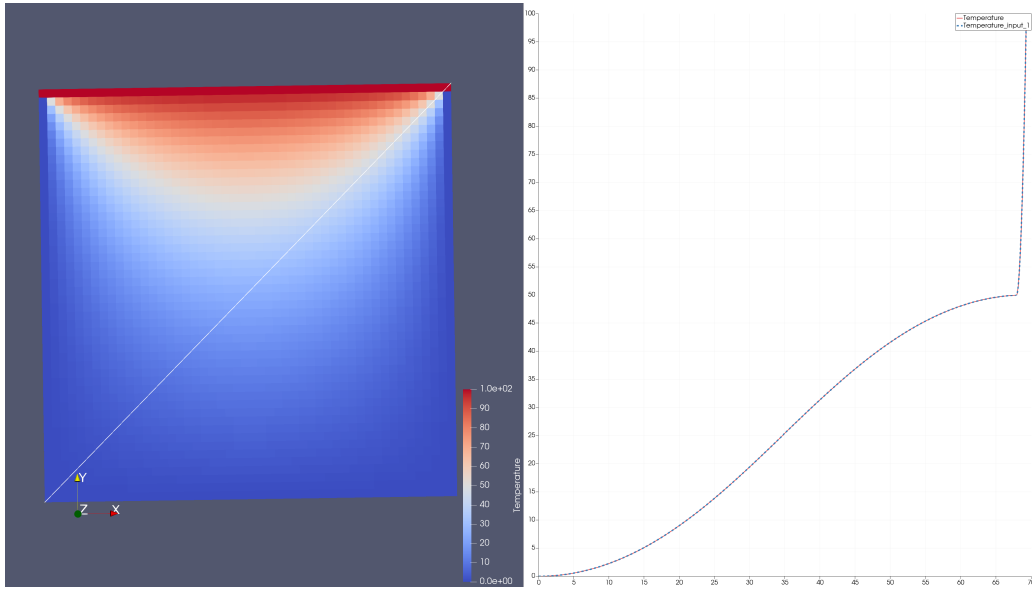


Figure 4.2: Multi-modal verification of the steady-state thermal field: (Left) 2D spatial distribution showing linear propagation; (Right) 1D center-line profile validating Dirichlet boundary condition matching.